CS498AE Final Project: Solving Hexcells

Adam Sitabkhan (asita2) May 2026

Code: `github.com/adamsitabkhan/hexcells-cs498ae`

Introduction

Hexcells is a hexagonal-grid logic puzzle in which every move can be deduced purely from the constraints visible on the board without guessing. Cells come in three kinds: yellow (unknown), blue (mine), and black (revealed, with a hint about its neighborhood). Hints come in three versions: a 6-cell ring around a black cell (*ZONE6*), an 18-cell hexagonal region around a blue cell (*ZONE18*), and column / diagonal counts along the puzzle border (*LINE*). Any hint may carry an adjacency modifier `{n}` for "the n mines are consecutive" or `-n-` for "no two of the n mines touch", which forces more interesting structure that pure Minesweeper or Sudoku constraints don't capture.

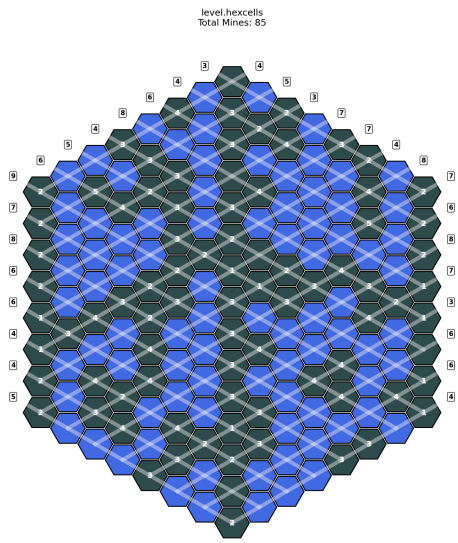

Figure 1: The 21-cell community puzzle *"Within Cells Interlinked (Easy)"*, rendered from its `.hexcells` definition. The board carries at least one of every hint type (ZONE6 on dark cells, ZONE18 on blue cells, LINE on the perimeter) and both adjacency modifiers (`{n}` for a contiguous run, `-n-` for forced separation). The full state is shown; a player initially sees only a small handful of revealed cells.

This project (Track C) compares four automated approaches to playing Hexcells: a Gurobi integer program, a Z3 SMT formulation that I improved down from 50–170 × slower than Gurobi to roughly 1.5 × slower with a single encoding change, two constraint-propagation heuristics (Single-Hint Propagation [SHP] and AC3), and the third-party Rust solver created by a Hexcells community member (Goguey) that I used as a reference. All four are evaluated on a library of 416 puzzles spanning 33–497 cells.

The biggest finding is that the choice of *encoding* within Z3 matters far more than the choice of *solver*. After replacing the natural $\sum_{c\in \text{ scope}} \mathbb{1}(x_c) = k$ cardinality constraint with Z3′s pseudo-Boolean primitive `PbEq`, Z3 closes most of its gap to Gurobi. We also find that classical CSP heuristics, while strong on easy puzzles in isolation (139 of 416 levels), contribute almost nothing once paired with either Z3 or Gurobi.

Background

A Hexcells puzzle is a constraint-satisfaction problem with N Boolean unknowns (mine vs. empty) and a mixture of cardinality equalities and global "runs" constraints. Two formulations we have explored apply directly.

Integer programming: Treat each cell as a binary variable $x_c \in \{0, 1\}$ and write each hint as a linear equality $\sum_{c\in \text{ scope}} x_c = k$. Gurobi solves the resulting binary program with branch-and-cut, LP relaxations at each node, presolve, and cutting-plane generation. For Hexcells the LP relaxation is unusually tight because of the large number of equality constraints, so the search trees are tiny.

SMT / Boolean cardinality: Treat each cell as a Boolean atom and encode hints either through linear-integer arithmetic (LIA) on $\mathbb{1}(b) =$ `If(b,1,0)` or through pseudo-Boolean primitives. Z3 has both: the arith theory encoding sends cardinality constraints through its arithmetic solver, whereas `PbEq` / `PbGe` dispatch to a specialised pseudo-Boolean propagator built on top of CDCL. This choice ended up having order-of-magnitude consequences on this problem class.

Constraint propagation. Two well-known CSP-style heuristics from the literature reduce most easy puzzles instantly without any search: single-hint propagation (SHP) and pairwise hint arc-consistency (AC3).

Method

Problem formulation

For every cell c, let $x_c \in \{0,1\}$ indicate that c is a mine. The total-mines constraint is $\sum_c x_c = M$. Each ZONE6 / ZONE18 / LINE hint contributes one scope constraint $\sum_{c \in \text{ scope}} x_c = k$. For hints with the TOGETHER (`{n}`) or SEPARATED (`-n-`) modifier and value $k \geq 2$ we expose the adjacency structure by introducing transition indicators $t_j = x_{c_{j-1}} \oplus x_{c_j}$ on the (cyclic, for ZONE6) sequence of scope cells, then add $\sum_j t_j = 2$ for TOGETHER (one contiguous run) or $\sum_j t_j \geq 4$ for SEPARATED (at least two disjoint runs). ZONE18 hints never carry a modifier.

Solver implementations

The IP module ([`project/gurobi_solver.py`]) builds a single Gurobi model with the binary variables, scope sums via `quicksum`, and four linear inequalities per transition variable to linearise the XOR. The SMT module ([`project/z3_solver.py`]) originally encoded every cardinality as `z3.Sum([z3.If(b,1,0) for b in scope]) == k`. Profiling a 169-cell medium puzzle revealed that Z3 spent over 97% of wall time in the per-cell push / pop loop of the incremental driver and that the `arith` theory was dominating its internal statistics (1.4M added equalities, 489k LIA bound propagations for a single iteration). A one-line change — replacing each constraint with `z3.PbEq([(b,1) for b in scope], k)` (and the analogous `PbGe` for SEPARATED) — drops the same puzzle from 12.92s to 0.05s. That single edit is the project's biggest improvement and is what we benchmark as "Z3 (PbEq)" below.

We also keep one ablation variant under `project/experiments/`: `z3_qffd` swaps `z3.Solver()` for `z3.SolverFor("QF_FD")`, Z3′s specialised finite-domain solver, on top of the same PbEq encoding. We benchmark it alongside the default Z3 module and discuss its effect in Section 5.

Heuristics

Two exact propagation heuristics are implemented in [`project/heuristics.py`]. *Single-Hint Propagation (SHP)* inspects each hint in isolation: if the remaining mine count equals zero, every uncovered cell in scope is empty; if it equals the number of uncovered cells, they are all mines. Combined with global mine-count accounting this solves all "obvious" deductions. *Arc Consistency 3 (AC3)* performs pairwise reasoning: for two hints A, B with overlapping scopes it tightens the mine-count interval of A minus B and B minus A using simple set arithmetic. Both heuristics are exact (they never reveal a wrong cell) and run in microseconds per iteration.

Rust reference-solver

The third-party solver in `external/reference-solver/` authored by Goguey and described as ranking puzzles by difficulty, a metric determined through computation by the solver. It uses forward constraint propagation, no SAT, IP, or branching. Each hint is materialised as a *multiverse*: an explicit list of `Layout`s, where a `Layout` is a map from cell-subsets to mine-counts. A ZONE6 with $k = 2$ ANYWHERE is one layout $\{a..f\} \to 2$; the TOGETHER modifier expands into one layout per contiguous block position; SEPARATED enumerates pivot positions, no auxiliary XOR variables. Inference is `Multiverse::merge` (intersection of two overlapping multiverses) followed by `Multiverse::invariants` (cells whose value agrees across every surviving layout). Depth is unbounded: trivial → compound → global merging of all visible constraints. The output's `Local(n)` / `Global(n)` tags report the merge depth at which each deduction was made.

Aspect	Gurobi IP	Z3 SMT (PbEq)	Rust multiverse
Cardinality $\sum_{c \in S} x_c = k$	`quicksum(scope) == k` (LP-relaxed)	`z3.PbEq([(v,1) for v in scope], k)`	one `Layout: {S → k}`
TOGETHER `{n}`	aux XOR vars t_j, `Sum(t)==2` linearized by 4 ineqs each	aux t_j, `z3.PbEq([(t,1)...], 2)`	`distribute_in_ring`: one layout per contiguous block
SEPARATED `-n-`	aux t_j, `Sum(t)>=4` linearized	aux t_j, `z3.PbGe([(t,1)...], 4)`	`distribute_separated`: enumerate pivot positions
Single-hint deduction	implicit (LP tightens bounds)	implicit (CDCL + PB propagation)	`invariants()` on one Layout (= SHP)
Multi-hint deduction	implicit (LP combines all simultaneously)	implicit (CDCL clause learning)	`Multiverse::merge` of adjacent hints, depth-extended
Search	branch-and-bound + cutting planes	CDCL backtracking	*none* — forward propagation only

Table 1: How each solver encodes Hexcells constraints and performs deductions. The Rust solver's headline structural advantage is the bottom row: it never branches.

Incremental solve loop

To mirror a player's perspective, the solver does not see hidden hints. The driver in [`project/solve.py`] interleaves heuristics and the chosen full solver in cycles. In each cycle the heuristics run to fixpoint, then the solver runs up to $M = 100$ "force-check" iterations: solve once, then for each unknown cell c test whether forcing x_c to the opposite value is infeasible. Any infeasible cell is added to the puzzle state and unlocks its hint. Cycles repeat until the puzzle is fully determined or no module makes progress.

Experiments

Setup. All experiments ran on an MacBook M4 Pro (Python 3.14, Z3 4.16, Gurobi 13.0, Rust 1.94). Each level was given a 60-second wall-clock budget, four levels in parallel, with results appended to a JSONL log. The levels library is the union of 219 community puzzles scraped from `r/hexcellslevels` and 197 procedurally generated levels produced by an level generator I wrote that reverse-engineers already-solved boards back into hint-minimal puzzle definitions, split into three tiers by cell count (*small* ≤ 100, *medium* 100–300, *large* > 300; totals $301 + 91 + 24 = 416$). 13 levels fail to parse for our Python parser and 17 for the Rust parser; these are counted as errors throughout.

Coverage and timing

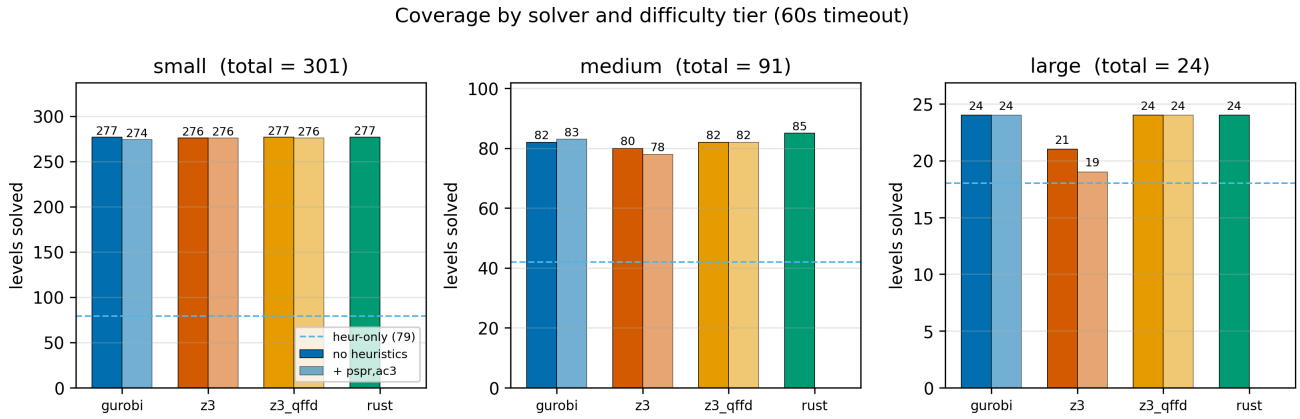

Figure 2: Levels solved within 60s, by solver and tier. Light bars overlay the with-heuristics variant on each solver. The dashed line is the heuristics-only baseline (SHP+AC3 with no full solver; registered as `pspr,ac3` in the code).

Coverage (Table 1) shows that the Rust reference-solver solves the most (386/416), Gurobi and Z3-QFFD tie at 383, Z3-PbEq is at 377 (losing 7 levels at the 60s timeout that the others find), and the heuristic-only baseline trails at 139. Adding heuristics to a full solver mildly reduces coverage for Z3-PbEq (377 → 373, lost on medium puzzles where the heuristic budget eats into solver time) and is a wash for Gurobi (383 → 381).

Configuration	small (/301)	medium (/91)	large (/24)	total
Gurobi IP	277	82	24	383
Gurobi IP + heur	274	83	24	381
Z3-PbEq	276	80	21	377
Z3-PbEq + heur	276	78	19	373
Z3-QFFD	277	82	24	383
Rust ref-solver [4]	277	85	24	386
Heuristics only (SHP+AC3)	79	42	18	139

Table 2: Levels solved per configuration. 60s timeout, 4 workers. Heuristics alone already account for $\frac{18}{24}$ of the large tier.

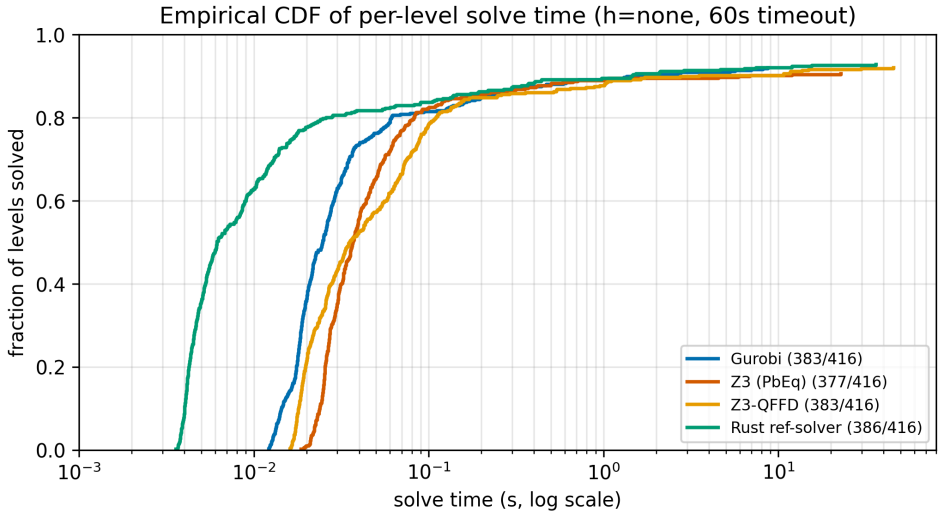

Figure 3: Empirical CDF of per-level solve times for the four no-heuristics configurations. The Rust solver dominates throughout: at 0.01s it has already finished half the corpus.

The CDF in Figure Figure 3 makes the per-instance ranking visible. The geometric mean of per-level $\frac{\text{cmp}}{\text{Gurobi}}$ ratios on the 375–382 levels both configurations solve is 1.43 for Z3-PbEq, 1.50 for Z3-QFFD, and 0.33 for the Rust solver. The Rust solver is roughly 3 × faster than Gurobi at the median; Z3 is roughly 1.5 × slower.

Where Z3 loses

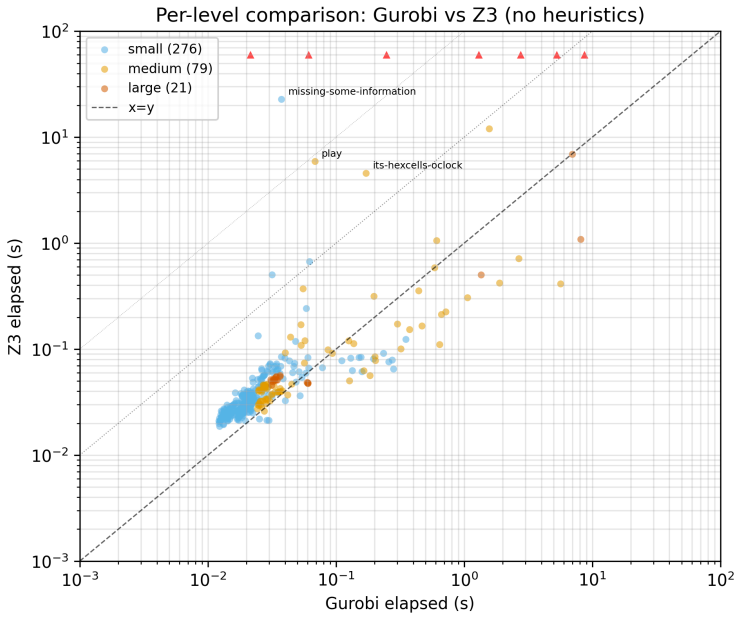

Figure 4: Per-level Gurobi (x) vs Z3-PbEq (y) elapsed time. Red triangles on the top axis are levels Z3 timed out on. Three labelled medium puzzles are the egregious outliers; two more — `modulo-mediumhard` and `sorcerer` — timed out and live on the upper axis.

The bulk of the scatter (Figure Figure 4) sits within a factor of 3 of the diagonal, which is the headline ratio. Five levels are cause a lot of struggles for Z3 and not for Gurobi: `modulo-mediumhard` and `sorcerer` time out (Gurobi finishes in 0.02s and 0.06s respectively), and `missing-some-information`, `play`, and `its-hexcells-oclock` take 5–23s where Gurobi takes < 0.2s. Profiling `modulo-mediumhard` (53 cells, 30 hints) under the Z3 driver reveals what is going wrong: a single outer force-check iteration takes 158s and runs 3.7M CDCL conflicts — roughly 75 k conflicts and 200 k decisions for **each** individual "is this cell forced?" SAT call, where the same query on a typical medium puzzle resolves in microseconds with under 100 conflicts. The level `sorcerer` is more extreme: 12.3M conflicts in one iteration. The bottleneck is not the encoding (it is the same `PbEq` encoding that solves the rest of the corpus in milliseconds) but CDCL search-tree explosion on these specific constraint structures. Gurobi's LP relaxation avoids this entirely.

Heuristic attribution

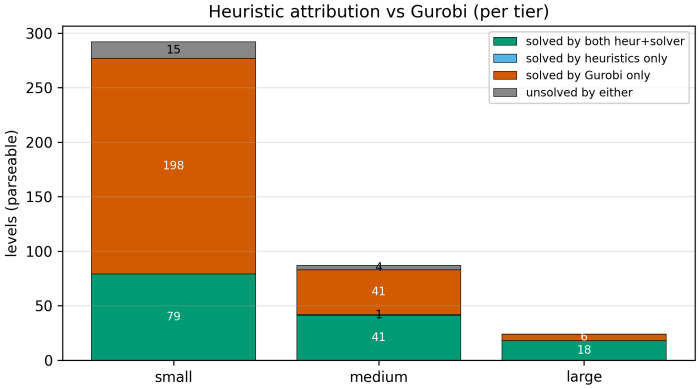

Figure 5: Levels solved by heuristics alone vs by Gurobi alone, per tier. On the small tier, only $\frac{79}{277}$ levels are heuristic-tractable; on the large tier, $\frac{18}{24}$ are. The "Gurobi only" category is where a full solver is doing real work; the "both" category is where heuristics suffice.

SHP + AC3 solve 79, 42, and 18 levels on small / medium / large respectively. Surprisingly, the large tier is the easiest for heuristics — the procedurally generated large levels in our corpus have a regular structure that local propagation exploits. The 244 levels where heuristics alone get stuck are precisely the ones a full solver needs to handle, and on those the heuristic prefix offers no speedup because the residual subproblem is itself the hard part.

Discussion

The encoding is the algorithm. The single most important change in this project was substituting `z3.PbEq` for `z3.Sum(If(...))`. Pre-change, Z3 was effectively unusable on large puzzles (229s on *"A Giant Scoop of Vanilla"*); post-change, the same level solves in under a second. No other engineering tweak improved performance by more than 10%.

Heuristics under a fast solver are a wash. I expected the heuristics-then-solver cycle to dominate, mirroring the role of presolve in an MIP pipeline. Empirically the heuristics solve the easy half of the library quickly, which is great, but they contribute essentially nothing once paired with Gurobi or Z3. The reason is that the remaining "hard" levels are hard *because* local propagation is insufficient; the heuristic prefix runs for milliseconds, finds nothing, and hands the same problem to the solver as before. In two cases, Z3 on the medium and large tiers, heuristics actively *hurt* coverage by consuming the per-iteration budget the solver would have spent on its force-check loop.

The Rust reference-solver is really fast. At a geomean $3\times$ faster than Gurobi and full coverage of the large tier, it sets a high bar for a Python implementation. As Table Table 1 shows, its advantage is structural: it never branches. Each `Multiverse` is a small set of layouts (ZONE6 ANYWHERE has 1 layout; ZONE18 ANYWHERE has 1; TOGETHER scales like $O(n)$); inference is a `BTreeMap` intersection followed by a per-cell agreement check. There is no LP relaxation, no CDCL bookkeeping, and no Python solver round-trip. The deduction primitive operates directly on cached integer sets. In this case, the problem class is narrow enough that explicit enumeration of feasible local arrangements is cheap.

Z3's five outliers are a CDCL pathology. The per-level profile above shows each pathological level runs millions of CDCL conflicts per outer iteration, three orders of magnitude more than a comparable typical puzzle of the same size. The encoding is fine; the search tree is the problem. Gurobi avoids this because its LP relaxation collapses these instances before branching is needed.

Limitations. All numbers are wall-clock on a single laptop with solver-internal threading disabled but four levels in parallel; per-level times are clean but absolute throughput is not. The level library mixes hand-crafted community puzzles with procedurally generated ones in a $5:4$ ratio, which biases the large-tier results toward puzzles amenable to heuristic propagation.

References

F. Goguey. *Hexcells Solver (Rust)*, `github.com/Ngoguey42/hexcells-solver`, 2023.
O. Prypin. *sixcells — Hexcells level format and Qt editor*, `github.com/oprypin/sixcells`, 2014–present.